

HearthMesh High Level Design

Global attributable emergency continuity mesh

Field	Value
Document version	0.1
Project status	Experimental pre-alpha
Date	24 September 2026
Project owner	Valerio Musella

Purpose

Define the target architecture and the deliberately narrower three-node MVP used to validate signed publication, replication, corruption detection and peer-assisted recovery.

Status statement

This design is not a production security assessment. The MVP demonstrates selected system behavior and requires protocol hardening, independent review and multi-host testing before real continuity data is entrusted to it.

1 Executive summary

HearthMesh is designed as a global, non-anonymous emergency continuity mesh. Independently operated nodes exchange readable public content that is signed, attributable, versioned and locally verified. A node can continue serving content already acquired when the original publisher, a peer or the wider Internet becomes unavailable.

The first implementation is intentionally smaller: three explicitly configured Go nodes on one Docker host. It proves the mechanics of signed HearthPacks, peer replication, local access, tamper detection and repair from a healthy replica. It does not yet provide global discovery, permissionless enrollment, private-cell confidentiality, key revocation or production-grade transport security.

The central architectural choice is accountability. The official design does not include onion routing, anonymous publication, payments, marketplaces or public encrypted blind storage. Runtime trust decisions use signatures, hashes and explicit operator policy; no AI model or inference service participates.

2 Scope and design principles

Goal	Architectural response
Continuity during disruption	Serve locally complete packs without a central cloud or reachable publisher.
Attributable public publishing	Bind each manifest to a persistent publisher key and visible operator record.
Integrity and provenance	Sign immutable manifests and verify each referenced blob by SHA-256.
Failure recovery	Pull a valid object from another approved peer after local corruption is detected.
Operator control	Apply local publisher, category, size, retention and replication policies.
AI independence	Use deterministic protocol rules with no model, agent or inference dependency.

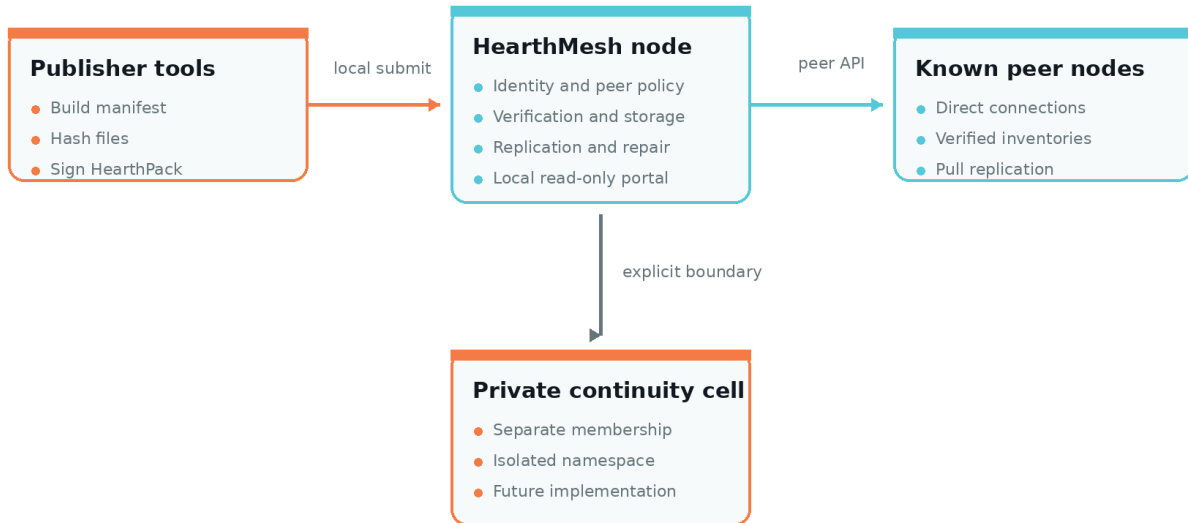
2.1 Explicit non-goals

- Anonymous participation, hidden publishers, onion routing or traffic-obfuscation guarantees.
- A global consensus protocol or universally authoritative latest version.
- Guaranteed truth, legality, safety, completeness or availability of published information.
- Real-time emergency dispatch or replacement of official emergency channels.
- Arbitrary anonymous uploads, payments, marketplaces or public blind storage.
- Production private cells, automated moderation or secure key recovery in the MVP.

3 Architecture overview

HearthMesh target architecture

Public continuity mesh with separately isolated private cells



Target architecture. The MVP implements a three-node allowlisted subset and does not implement private-cell confidentiality.

Figure 1. Target architecture and the private-cell isolation boundary.

The target system uses the same core node software for the public mesh and future private cells, but the two spaces must not share discovery, indexes, membership or storage by default. For early private-cell work, separate processes and volumes are safer than unproven in-process multi-tenancy.

Component	Primary responsibility
Identity manager	Loads node identity, publisher trust records and pinned peer identities.
Peer transport	Connects directly to configured peers and applies timeouts and transfer limits.
Pack verifier	Checks manifest structure, publisher signature, paths, media policy and blob hashes.
Content store	Stores immutable manifests and content-addressed blobs with atomic promotion.
Replication scheduler	Polls known peers and pulls locally eligible missing or damaged objects.
Integrity scanner	Recomputes hashes, quarantines corrupt objects and requests repair.
Local portal	Shows available packs, versions, provenance and node status without executing pack content.

Component	Primary responsibility
Publisher tools	Builds and signs HearthPacks through a local administrative path.

4 Trust model and boundaries

A valid signature proves control of a publisher key; it does not prove that the material is correct. An authenticated peer is identifiable; it is not automatically honest. HearthMesh therefore keeps publisher trust, peer admission and content verification as separate decisions.

Boundary	Risk	Required control
Operator to local administration	Policy deletion or misuse of signing material	Local binding, restricted permissions and separated publisher keys.
Peer to transport	False inventory, resource exhaustion or stale content	Pinned identities, limits, pull replication and backoff.
Received bytes to storage	Malformed, oversized or altered objects	Staging, schema checks, signature verification and digest verification.
Publisher to reader	Authentic but incorrect or harmful content	Visible attribution, local trust tier and operator moderation.
Public mesh to private cell	Metadata or content leakage	Separate membership, namespace, storage, discovery and keys.
Host to node	Key and data compromise	Host hardening; acknowledge that host compromise defeats local guarantees.

4.1 Identity and trust tiers

Node identity, publisher identity and accountable operator identity are distinct. An Ed25519 key alone creates a stable pseudonym, not verified real-world identity. Public enrollment must therefore attach an operator name or organization, contact route and explicit verification status to the cryptographic identity.

Tier	Meaning	Default replication treatment
Blocked	Locally prohibited publisher	Reject and do not advertise.
Unrecognized	Valid signature without operator approval	Do not replicate automatically.

Tier	Meaning	Default replication treatment
Enrolled	Accountable publisher record approved locally	Eligible under local content policy.
Endorsed	Approved for a defined role or information domain	Display endorsement and its exact scope.

5 HearthPack data model

A HearthPack is an immutable signed manifest plus the immutable blobs referenced by that manifest. The pack identifier is derived from canonical manifest bytes; a blob identifier is derived from the exact blob bytes.

Element	Required content
Protocol metadata	Protocol version, manifest schema version and algorithm identifiers.
Publisher	Publisher identifier, public-key reference and signature envelope.
Publication identity	Logical publication ID, revision and optional predecessor manifest digest.
Descriptive metadata	Title, summary, language, topic tags, geography, license and review-by date.
Entries	Normalized path, media type, byte size and SHA-256 digest for every file.
Classification	Public in MVP; private classifications are rejected until private cells exist.

Protocol version 1 must freeze canonical encoding and signature inputs before interoperability is claimed. Ordinary JSON map serialization is not sufficient unless field ordering, whitespace, duplicate-field handling and numeric representations are specified and tested.

```
blob_id = SHA-256(blob bytes)
pack_id = SHA-256(canonical manifest bytes)
```

6 Networking and interfaces

The MVP uses direct, explicitly configured peer addresses and periodic pull replication. The target design should use TLS 1.3 mutual authentication with peer identities pinned to expected node keys. Bootstrap peers provide initial addresses; they are not authorities.

Interface	Purpose	Exposure
GET /v1/node	Signed node descriptor and protocol capabilities	Approved peers
GET /v1/inventory	Bounded pages of complete shareable pack IDs	Approved peers
GET /v1/manifests/{pack_id}	Signed manifest envelope	Approved peers
GET /v1/blobs/{blob_id}	Verified immutable blob bytes	Approved peers
Local portal	Browse, verify and download installed public material	Local network by policy
Administrative API	Publish, trust, retention and maintenance actions	Local host only

The target protocol requires bounded response sizes, timeouts, concurrency limits, retry backoff and jitter. Nodes must never dial arbitrary endpoints embedded in a manifest or automatically admit transitively advertised peers.

7 Core system flows

7.1 Publish

1. The local operator selects files and publication metadata.
2. Publisher tooling validates paths, sizes and allowed media types.
3. Files are hashed and a canonical manifest is signed.
4. The local node independently verifies the pack and publisher policy.
5. Objects move from staging into immutable storage only after verification.
6. A complete pack becomes visible locally and eligible for peer inventory.

7.2 Replicate

1. A node polls an approved peer inventory.
2. It fetches an unfamiliar signed manifest and evaluates local admission policy.
3. Missing blobs are pulled into bounded staging storage.
4. Every size and digest is verified before the pack is committed.
5. Only complete packs appear in the local portal or outgoing inventory.

7.3 Detect and repair corruption

MVP failure and recovery flow



Repair succeeds only while another reachable peer retains bytes matching the signed manifest.

Figure 2. The MVP recovery case: publisher loss, damaged replica and verified repair.

On a digest mismatch, the object is quarantined and dependent packs become unavailable. The scheduler asks other configured peers for the expected digest, verifies replacement bytes and installs them atomically. Hashes detect damage but cannot reconstruct data; recovery fails if every correct replica has been lost.

8 Public mesh and private continuity cells

Property	Global public mesh	Private continuity cell
Purpose	Broadly distributable emergency and continuity knowledge	Restricted operational or personal material
Content visibility	Readable public content	Authorized members only
Publisher model	Attributable publisher with local trust status	Known membership and cell-scoped authorization
Discovery	Future global federation; allowlisted in MVP	Isolated and not publicly advertised
Storage	Public immutable objects	Separate encrypted store and index
MVP status	Partially demonstrated	Architecture placeholder only

A private cell is not a hidden anonymous channel. It requires explicit membership, isolated peer lists, authenticated encrypted transport, data-at-rest policy and tested key management. Revoking membership cannot erase copies already obtained.

9 Security and misuse resistance

The MVP should demonstrate cryptographic integrity, not broad security. It must reject malformed manifests, unsafe paths, altered blobs and unsupported algorithms. Publisher-supplied code must never execute in the portal, and the node must not automatically extract archives or fetch embedded external resources.

Threat	MVP response	Remaining exposure
Dishonest publisher	Visible signature and local trust policy	Signature cannot establish truth.
Compromised publisher key	Manual block configuration	Disconnected nodes may not learn the revocation.
Malicious peer	Allowlist, pull transfer and limits	Approved peer can suppress or waste resources.
Corrupt local object	Digest scan, quarantine and peer repair	No repair if all valid copies are lost.
Path traversal	Normalize and reject unsafe entry paths	Parser defects still require testing and fuzzing.
Active file content	MVP allows a small passive format set	A valid file may still exploit an external viewer.
Sybil enrollment	Manual enrollment in MVP	Global admission remains unsolved.
Traffic analysis	No concealment is attempted	Peer relationships and timing remain visible.

9.1 Misuse constraints

- No onion routing, hidden services, anonymous publisher mode or traffic-obfuscation claim.
- No public encrypted blind-storage service in the official network.
- No payments, marketplace, anonymous messaging or arbitrary unauthenticated upload endpoint.
- Public packs remain readable and attributable; node operators choose what they replicate.
- Moderation and deletion are local. Global recall of replicated material cannot be guaranteed.

These choices reduce darknet-like utility but cannot prevent modification or misuse of open-source code. The project should describe enforceable protocol behavior precisely and avoid claiming that file-extension checks can prove the absence of hidden or encrypted data.

10 Failure behavior

Failure	Required behavior
One peer unavailable	Continue local service and retry another approved peer.
All peers unavailable	Serve complete local packs and show disconnected status.
Interrupted transfer	Keep bounded staging state; never expose an incomplete pack.
Digest mismatch	Quarantine the object, record the event and request another source.
Invalid signature	Reject the manifest with a visible reason.
Disk full	Stop acquisition safely and preserve installed content where possible.
Conflicting revisions	Retain and show both branches; do not resolve silently by timestamp.
Lost node key	Require explicit reenrollment under a new identity.
All correct copies lost	Declare content unavailable; do not imply recoverability.
MVP Docker host lost	All three logical nodes may fail because they share one physical host.

11 Deployment and operations

Docker Compose starts three logical Go nodes with distinct identities, ports, configurations and persistent volumes. This is reproducible and sufficient for behavior testing, but all nodes share one host, disk, power source and network path. A meaningful resilience pilot requires independent machines and failure domains.

Operational signal	Interpretation
Process healthy	The node process is running and local checks pass.
Content ready	At least the required locally pinned packs are complete and verified.
Mesh degraded	One or more peers are unavailable; local continuity may still be intact.
Pack quarantined	A signature, schema or digest verification failed.
Storage pressure	Acquisition may stop before installed pinned content is affected.

Logs and metrics should cover peer reachability, pack states, replication retries, bytes transferred, verification failures, integrity-scan coverage, disk quotas and protocol versions. Private content and unnecessary reader identifiers must not be logged.

12 MVP implementation

Capability	MVP position
Runtime	Single Go binary per node
Topology	Three allowlisted nodes with explicit bootstrap peers
Identity	Persistent Ed25519 node and publisher material
Integrity	Signed manifest plus SHA-256 file verification
Replication	Direct peer polling and pull transfer
Recovery	Detect modified bytes and obtain a valid peer copy
Access	Local read-only web view and status endpoints
Packaging	Docker Compose and scripted demonstration
Status	Experimental pre-alpha; not production-secure

12.1 Demonstration acceptance

1. Node A publishes a signed sample HearthPack.
2. Nodes B and C replicate and verify the same pack.
3. Node A is stopped and no longer serves as publisher.
4. A stored file on node B is deliberately modified.
5. Node B detects that the file digest does not match the signed manifest.
6. Node B retrieves the expected bytes from node C and verifies them before restoration.
7. The recovered pack remains available through the local portal.

Acceptance was completed on 24 September 2026 on a Windows x86-64 host. All four Go tests passed, the Windows binary built successfully, and the three-node Docker drill demonstrated publication, replication, continued availability after node A stopped, corruption detection on node B and repair from node C. This is single-host functional evidence, not an independent security audit or a multi-host resilience test. Continuous integration must repeat tests and builds for ongoing changes.

13 Architecture decisions

Decision	Rationale
Immutable content addressing	Makes stored objects independently verifiable and avoids silent mutation.
Separate publisher and node identities	Publishing authority should not be implied by transport participation.
Direct identifiable peers	Supports accountability and intentionally rejects anonymity infrastructure.
Pull replication	Prevents unsolicited peers from pushing arbitrary content into storage.
Local policy	Signing does not create a storage obligation for every node.
No global consensus	The system can preserve conflicting branches without pretending to select truth.
No AI at runtime	Trust decisions remain deterministic, inspectable and service-independent.
Separate private-cell process	Avoids claiming safe multi-tenancy before isolation is designed and tested.

14 Roadmap

Phase	Outcome	Exit evidence
1 Protocol specification	Canonical encoding, IDs, limits and test vectors	Independent implementations produce identical IDs and signatures.
2 Three-node MVP	Publication, replication, portal, tamper detection and repair	Automated tests plus a recorded repeatable Docker drill.
3 Multi-host pilot	Independent operators and physical failure domains	Measured recovery during network and publisher loss.
4 Security hardening	Rotation, revocation, fuzzing, abuse response and dependency review	Independent assessment and tracked remediation.
5 Private cells	Membership, isolation, encryption and key lifecycle	Confidentiality and revocation-limit tests.
6 Broader federation	Scalable discovery and inventory exchange	Pilot evidence without a mandatory central coordinator.

15 Principal limitations

- The MVP proves a local three-node behavior, not an operational global mesh.
- Three containers on one machine do not provide hardware, power or geographic resilience.
- A valid signature proves key control and byte integrity, not factual correctness.
- Manual enrollment reduces the initial attack surface but does not solve global identity or Sybil resistance.
- Key rotation, revocation, recovery and publisher withdrawal require protocol and governance work.
- Public replication makes universal deletion impossible once another party retains a copy.
- Private continuity cells are an architectural direction, not an implemented confidentiality feature.
- Security claims require independent review, adversarial testing and operational evidence.

The correct pre-alpha claim is narrow: HearthMesh demonstrates how attributable signed content can remain locally available, detect alteration and recover from another known peer without relying on the original publisher.